

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

طراحی و پیاده سازی یک سامانه هوشمند برای تطبیق، تنقیح و تحلیل قوانین فارسی با استفاده از مدل های زبانی بزرگ

نام دانشجو:	علی دهقان زاده
شماره دانشجویی:	۸۱۰۱۰۱۴۲۳
نام درس / استاد:	استاد هشام فیلی

تعهدنامه اصالت اثر

باسمه تعالی

اینجانب علی دهقان زاده تأیید می کنم که مطالب مندرج در این پایان نامه حاصل تلاش اینجانب است و به دستاوردهای پژوهشی دیگران که در این نوشته از آنها استفاده شده است مطابق مقررات ارجاع گردیده است. این پایان نامه قبلاً برای احراز هیچ مدرک هم سطح یا بالاتر ارائه نشده است.

کلیه حقوق مادی و معنوی این اثر متعلق به دانشکده فنی دانشگاه تهران می باشد.

نام و نام خانوادگی دانشجو : علی دهقان زاده

امضای دانشجو :

تقدیم به مادرم که همیشه کنارم بوده

فهرست مطالب

۱.	فصل اول: مقدمه
۱/۱.	بیان مسئله
۱/۲.	هدف تحقیق
۱/۳.	ساختار پایان نامه
۲.	فصل دوم: مقدمه ای بر بعضی مفاهیم
۲/۱.	مقدمه
۲/۲.	پردازش زبان طبیعی و مدل های زبانی
۲/۲/۱.	مدل های زبانی
۲/۲/۲.	مدل های زبانی بزرگ
۲/۲/۳.	پردازش زبان طبیعی در متون حقوقی
۲/۳.	بازیابی اطلاعات از متون حقوقی
۲/۳/۱.	بازیابی واژگانی با روش BM25
۲/۳/۲.	جست و جوی معنایی و بردارهای تعبیه شده
۲/۳/۳.	بازرتبه بندی نتایج
۲/۴.	تولید تقویت شده با بازیابی
۲/۵.	تعاریف کوتاه دیگر
۲/۵/۱.	شاخص بندی (indexing)
۲/۵/۲.	Django
۲/۵/۳.	Postgres
۲/۵/۴.	Milvus
۲/۵/۵.	RRF (Reciprocal Rank Fusion)
۳.	فصل سوم: طراحی سامانه
۳/۱.	مقدمه کوتاه
۳/۲.	قسمت استخراج متن
۳/۳.	معماری ساختار داده های حقوقی
۳/۴.	معماری قسمت بازیابی اطلاعات
۳/۵.	معماری قسمت تولید پاسخ مبتنی بر بازیابی
۴.	فصل چهارم: اجرا
۴/۱.	استخراج داده
۴/۲.	آماده سازی زیرساخت
۴/۳.	انتقال و آماده سازی داده استخراج شده
۴/۴.	اجرای نهایی
۵.	فصل پنجم: جمع بندی
۵/۱.	نتیجه گیری
۵/۲.	محدودیت ها
۶.	منابع و مراجع

۱. مقدمه

۱-۱. بیان مسئله

یکی از مهم ترین مسائل در نظام های اطلاعات حقوقی، این است که هیچ ابزار کارآمدی برای بررسی قوانین موجود در حوزه های حقوقی وجود ندارد که از نظر دقت و سرعت قابل اتکا باشد. اکثر روش های جستجو، متون پیچیده اسناد حقوقی را متوجه نمی شوند و پاسخی که به سوال کاربر داده می شود، به اسنادی اشاره می کنند که هیچ ارتباطی به موضوع مورد بحث در سوال نیستند.

البته باید این را نیز در نظر گرفت که کاربرانی که این سوالات حقوقی را می پرسند، از دانش لغوی و حقوقی کامل برخوردار نیستند که سوال را دقیق بپرسند. متون حقوقی معمولاً پیچیدگی های خاصی را همراه دارند؛ از جمله ارتباطات میان احکام و مصوبات مختلف (مثل نسخ، اصلاحیه و ...)، استفاده از کلمات فوق تخصصی و معتبر بودن یا نبودن سند در گذر زمان (به خاطر بخشنامه ها و مصوبات بعد از آن). بنابراین یک جستجوی کلمه محور که متن حقوقی را از لحاظ معنایی نمی تواند درک کند، برای چنین حوزه ای کافی نیست.

به عنوان مثال، این متن ماده ۸۴۳ قانون مدنی است:

«ماده ۸۴۳ - وصیت به زیاده بر ثلث ترکه، نافذ نیست. مگر به اجازه وراثت. و اگر بعض از ورثه، اجازه کند فقط نسبت به سهم او نافذ است.»

این متن شامل تعداد بسیار زیادی واژه عربی و حقوقی سخت است که برای یک کاربر عادی ممکن است فهم آن دشوار باشد، ولی معنای ساده آن اینگونه است:

«هیچ کس نمی تواند وصیت کند که بعد از مرگش، بیشتر از یک سوم (ثلث) اموالش به فرد خاصی برسد یا خرج کار خاصی شود. اگر بیشتر از یک سوم را وصیت کند، آن وصیت تنها در صورتی اجرا می شود که اعضای خانواده (ورثه) با آن موافقت کنند. اگر فقط بعضی از آن ها موافقت کنند، وصیت فقط از سهم ارث همان افراد کسر می شود.»

۱-۲. هدف تحقیق

در پروژه حاضر، سعی شده سامانه ای هوشمند مانند chatgpt بالا بیاید که تخصصش پاسخ به سوالات و شبهات قانونی کاربران است. تفاوت این سامانه با سامانه هایی مانند chatgpt این است که مدل تنها در راستای مباحث حقوقی و مسائل قانونی و قضایی سوالات را پاسخ دهد و مستندات فراهم شده برای مدل همگی مرتبط به همین حوزه ها هستند. این موضوع کمک می کند سوالات حقوقی با دقت بالاتری بررسی شده و پاسخ قابل استناد تری تحویل داده شود.

از طرفی داده های حقوقی به صورت ساختارمند و طبق مدل های مختلف اسناد (مصوبه، بخشنامه، قانون، مقررات، ...) و عناصر حقوقی (ماده، تبصره، بند، ...) در سامانه نگهداری می شود. این ساختار به سیستم اجازه

می دهد که در کنار استخراج صرف متن عنصر حقوقی، به بخش ها، مواد، تبصره ها و ریشه های قوانین نیز دسترسی پیدا کند.

هدف اصلی این تحقیق، طراحی و پیاده سازی یک سامانه پاسخ گویی هوشمند به پرسش های حقوقی است که بتواند. اسناد و مقررات حقوقی جمع آوری شده را ساختاردهی کند، به صورت مؤثر، اطلاعات مرتبط با پرسش کاربر را بازیابی نماید، پاسخ های حقوقی را بر اساس مدارک معتبر و منابع موجود ارائه دهد (به گونه ای که حتی از نظر زمانی نیز قابل استناد باشد)، و در کنار ارائه پاسخ، استنادات لازم را به متن و ماده های مرتبط اعمال کند.

در این سامانه به دنبال آموزش یک مدل زبانی مخصوص که تنها و تنها متون حقوقی را متوجه شود نیستیم. این سامانه از یک مدل هوش مصنوعی آماده در بازار استفاده می کند، ولی با این تفاوت که سامانه سوال کاربر را در متون حقوقی موجود در پایگاه داده سامانه جستجو می کند، نتایج مرتبط باز می گرداند و به مدل می دهد که با توجه به متون داده شده، مفهوم قانون ساده سازی شده و به پرسش کاربر پاسخ داده شود.

نکته مهم: در قسمت های مختلف این گزارش از عبارت «متون حقوقی» استفاده شده است ولی تمامی قوانین، مقررات، بخشنامه ها و ...، و همچنین در تمامی حوزه هاس و نه فقط حوزه حقوق را در بر می گیرد.

۳-۱. ساختار گزارش

این گزارش در شش فصل تنظیم شده است. در فصل اول، مسئله، اهداف و کلیات پروژه بیان می شود. فصل دوم به معرفی مفاهیم و روش های مرتبط با پردازش زبان طبیعی، بازیابی اطلاعات و مدل های زبانی اختصاص دارد. در فصل سوم، معماری و طراحی سامانه ارائه می شود. در فصل چهارم، نتیجه گیری و محدودیت های پروژه بررسی می شوند. در نهایت، فصل پنجم شامل منابع و مراجع مورد استفاده در این پروژه است.

فصل ۲: مفاهیم پایه و پیش زمینه

۲-۱. مقدمه

در سال‌های اخیر، پیشرفت روش‌های هوش مصنوعی و پردازش زبان طبیعی، امکان پردازش و تحلیل حجم زیادی از داده‌های متنی را فراهم کرده است. ظهور مدل‌های زبانی بزرگ که متن‌های مختلف را به راحتی خوانده و متوجه می‌شوند، یکی از بزرگترین انقلاب‌های اتفاق افتاده در هوش مصنوعی است. همچنین چندین پایگاه داده و چندین روش مختلف برای بازیابی اطلاعات به وجود آمده است که کار استخراج متن از پایگاه داده را برای سامانه آسان‌تر و سریع‌تر می‌کند.

در این فصل، مفاهیم و فناوری‌های مورد استفاده در طراحی سامانه حاضر بررسی می‌شوند. ابتدا مفاهیم مرتبط با پردازش زبان طبیعی و مدل‌های زبانی معرفی شده و سپس مفاهیم مربوط به بازیابی اطلاعات، جستجوی معنایی، باز رتبه بندی (reranking) و تولید تقویت شده با بازیابی (RAG) بررسی خواهند شد. این مفاهیم، مبنای نظری معماری سامانه پیشنهادی در فصل‌های بعدی را تشکیل می‌دهند.

۲-۲. پردازش زبان طبیعی و مدل‌های زبانی

پردازش زبان طبیعی (Natural Language Processing یا NLP) یکی از انقلابی‌ترین شاخه‌های هوش مصنوعی است که به مطالعه و توسعه روش‌هایی برای پردازش، تحلیل و تولید زبان انسان توسط رایانه می‌پردازد. هدف اصلی این حوزه، ایجاد امکان تعامل مؤثر میان انسان و سامانه‌های کامپیوتری از طریق زبان طبیعی و استخراج اطلاعات و معنا از متون و گفتار است.

متون زبان طبیعی برخلاف داده‌های ساختاریافته، دارای ساختار صریح و ثابتی نیستند و یک مفهوم واحد می‌تواند با استفاده از عبارت‌ها و کلمات متفاوت بیان شود. برای مثال، در یک پرسش حقوقی ممکن است کاربر مفهوم مشخصی را با واژه‌هایی بیان کند که عیناً در متن قانون وجود ندارند. بنابراین، سامانه‌های پردازش زبان طبیعی باید بتوانند علاوه بر تطبیق واژه‌ها، تا حد امکان ارتباط معنایی میان عبارات را نیز در نظر بگیرند.

تعاریف زیر شاید برای بسیاری از خوانندگان در حوزه مهندسی کامپیوتر آشنا باشد، ولی از این جهت که این حوزه‌ها حوزه تخصصی کارشناسی ارشد مهندسی کامپیوتر است و نه کارشناسی، اینجا این مفاهیم آورده شده است.

۲-۲-۱. مدل‌های زبانی

مدل زبانی (Language Model) مدلی است که با استفاده از الگوهای موجود در داده‌های متنی، احتمال وقوع توالی‌های مختلف از کلمات (یا در زبان کامپیوتری، توکن‌ها یا به انگلیسی token) را یاد می‌گیرد. به بیان ساده، مدل زبانی می‌تواند بر اساس متن مشاهده شده، احتمال ادامه یا ارتباط بخش‌های مختلف یک متن را تخمین بزند.

در گذشته مدل های زبانی اولیه عمدتاً بر اساس روش های آماری و احتمال وقوع کلمات در کنار یکدیگر ساخته می شدند. با پیشرفت یادگیری عمیق، شبکه های عصبی جایگزین بسیاری از این روش ها شدند و مدل هایی با قابلیت درک بهتر وابستگی های موجود در متن توسعه یافتند.

یکی از مراحل مهم در توسعه مدل های زبانی، استفاده از معماری Transformer است. این معماری با استفاده از سازوکار Attention امکان بررسی ارتباط میان بخش های مختلف یک متن را فراهم می کند و در مقایسه با بسیاری از روش های پیشین، توانایی بهتری در پردازش وابستگی های دور در متن دارد. از آنجایی که وارد طراحی این مدل ها نشده ایم، بیشتر وارد جزئیات نمی شویم.

۲-۲-۲. مدل های زبانی بزرگ

مدل های زبانی بزرگ (Large Language Models یا LLMs) دسته ای از مدل های زبانی هستند که با استفاده از حجم بسیار زیادی از داده های متنی و تعداد زیادی پارامتر آموزش داده می شوند. افزایش مقیاس مدل و داده های آموزشی باعث شده است این مدل ها بتوانند وظایف متنوعی مانند تولید متن، پاسخگویی به پرسش ها، خلاصه سازی، استخراج اطلاعات و تحلیل متون را انجام دهند.

مدل های زبانی بزرگ معمولاً ابتدا طی فرایند پیش آموزش (Pre-training) بر روی حجم زیادی از متن آموزش داده می شوند. در این مرحله، مدل با مشاهده نمونه های مختلف زبان، الگوهای زبانی و ارتباط میان کلمات (یا همان token ها) را یاد می گیرد. پس از آن، می توان مدل را برای کاربرد های خاص، از جمله پاسخ گویی به پرسش یا پردازش متون تخصصی، مورد استفاده یا تنظیم قرار داد.

با وجود قابلیت های گسترده، مدل های زبانی بزرگ محدودیت هایی نیز دارند. یکی از مهم ترین این محدودیت ها، امکان تولید اطلاعات نادرست یا غیر مستند است (مشکلی که در حوزه کامپیوتر به نام توهم یا hallucination شناخته می شود). این مسئله در حوزه حقوق اهمیت بیشتری دارد، زیرا یک پاسخ ظاهراً صحیح اما فاقد مبنای قانونی می تواند اعتماد طرفین یک قرارداد را به هم بزند. در بعضی موارد ترجمه نادقیق کلمات به زبان های مختلف در قرارداد های بین المللی می تواند روابط دیپلماسی را در هم بشکند.

مدل های زبانی که بالاتر توضیح داده شد، الزامی ندارد که حتماً زبانی را که در زندگی روزمره استفاده می کنیم یاد بگیرد. مدل هایی نیز ساخته شده اند که صرفاً برای کد یک زبان خاص طراحی شده اند که یک برنامه را (به عنوان مثال زبان پایتون) بهینه بنویسد و الگوریتم های سریعتری در آن استفاده کند. این مدل ها ممکن است متن فارسی، انگلیسی یا هر زبان دیگر را متوجه نشوند، چون یاد گرفته اند که به همه چیز به شکل کد نگاه کنند.

۲-۲-۳. پردازش زبان طبیعی

پردازش زبان طبیعی (Natural Language Processing یا NLP) یکی از شاخه های هوش مصنوعی است که به پردازش و تحلیل زبان انسان توسط کامپیوتر می پردازد. زبان انسان برخلاف داده های ساختاریافته، معمولاً شکل مشخص و ثابتی ندارد و می تواند شامل ابهام، تفاوت در نحوه بیان و وابستگی میان بخش های

مختلف جمله باشد. هدف پردازش زبان طبیعی این است که کامپیوتر بتواند این نوع داده‌ها را پردازش کرده و اطلاعات و الگوهای موجود در آن‌ها را استخراج کند.

یکی از کارهایی که پردازش متن را راحت تر می کند، تبدیل آنها به آرایه ای (بهتر بگوییم، برداری) از اعداد است که کلمات با معانی نزدیک به هم (مانند فرش و موکت، میوه و انگور، ...) بردارهای نزدیک به هم داشته باشند. منظور از اینکه بردار هایشان نزدیک به هم باشد، این است که فاصله این دو بردار (که روش‌های متعددی برای اندازه‌گیری فاصله بین دو بردار اتخاذ می شود) از حدی بیشتر نباشد (یا بهتر بگوییم، فاصله اش کمتر از فاصله دو کلمه کاملاً نامربوط باشد).

متون حقوقی ویژگی‌هایی دارند که پردازش آنها را نسبت به متون عمومی دشوارتر می کند. استفاده از اصطلاحات تخصصی، ساختار سلسله‌مراتبی قوانین، اهمیت علائم نگارشی، وابستگی میان مواد و تبصره‌ها، ارجاع به سایر اسناد و وجود نسخه‌های مختلف یک مقرر از جمله این ویژگی‌ها هستند.

علاوه بر این، در متون حقوقی تفاوت میان دو عبارت ظاهراً مشابه ممکن است از نظر حقوقی اهمیت زیادی داشته باشد. بنابراین، سامانه حقوقی باید بتواند علاوه بر پردازش محتوای متنی، ساختار و اطلاعات مرتبط با هر بخش از سند را نیز در نظر بگیرد.

در زبان فارسی نیز چالش‌هایی مانند تنوع نوشتاری، شکل‌های مختلف فاصله و نیم فاصله، وجود نویسه‌های متفاوت برای برخی حروف و پیچیدگی ساختاری متون می تواند بر عملکرد روش‌های پردازش زبان طبیعی تأثیر بگذارد. از این رو، پیش پردازش و نرمال سازی مناسب متن و استفاده از مدل‌هایی که توانایی پردازش زبان فارسی را دارند، از عوامل مهم در طراحی سامانه‌های حقوقی فارسی محسوب می شوند.

۳-۲. بازیابی اطلاعات در متون حقوقی

یکی از کارهایی که می توان انجام داد که تا جای ممکن بار از دوش مدل‌های زبانی برداشته شود (که مشکلاتی مانند توهم زدن را کاهش دهد)، استفاده از ابزارهای بازیابی اطلاعات است (توجه شود در اینجا منظور از بازیابی، retrieval است و نه recovery). این ابزارها در پایگاه داده اطلاعات را جستجو می کنند و می توان نتایج جستجوی مرتبط را به مدل‌ها داد تا مطمئن شد مدل‌ها استناد به یک قانون خودساخته نمی کنند.

در یک سامانه حقوقی، اهمیت بازیابی اطلاعات بیشتر است؛ زیرا حجم اسناد و مقررات می تواند بسیار زیاد باشد و پاسخ به یک پرسش ممکن است به یک ماده یا مجموعه‌ای از مواد قانونی وابسته باشد. در چنین شرایطی، جستجوی ساده بر اساس تطبیق کلمات کلیدی همیشه (شاید هم هیچوقت) نمی تواند مناسب ترین مواد را پیدا کند. در مثالی که در قسمت مقدمه زده شد، اگر عبارتی که کاربر جستجو می کند از لحاظ لغوی کاملاً متفاوت با آن چیزی باشد که در متن نوشته شده، نمی توان تضمین کرد که نتیجه جستجو (حتی ذره‌ای) مرتبط با سوال پرسیده شده باشد. در چند بخش زیر، مکانیزم‌های استفاده شده در سامانه برای جستجو و بازیابی پرسش آورده شده است.

۱-۳-۲. بازیابی واژگانی با روش BM25

یکی از روش های متداول در بازیابی اطلاعات، جستجوی واژگانی است (درست است که در بالا گفته شد کافی نیست، ولی سامانه برای مواقعی نیز طراحی شده است که یک کاربر حرفه ای در حوزه حقوقی نیز بتواند از آن بهره ببرد). در این روش، ارتباط میان پرسش و سند بر اساس حضور و اهمیت خود واژه های موجود در آن ها تعیین می شود.

یکی از الگوریتم های شناخته شده در این زمینه BM25 است. این الگوریتم نسخه بهبود یافته ای از روش های مبتنی بر TF-IDF محسوب می شود و تلاش می کند میزان ارتباط یک سند با یک پرسش را با در نظر گرفتن چند متغیر مختلف محاسبه کند. مهم ترین این متغیر ها عبارتند از:

- تعداد دفعات تکرار یک واژه در یک سند (بخش TF که می شود Term Frequency)
- میزان اهمیت و نادر بودن واژه در مجموعه اسناد (بخش IDF که می شود Inverse Document Frequency)
- طول سند نسبت به طول متوسط اسناد موجود

فرمول کلی BM25 به صورت زیر است:

$$Score(D, Q) = \sum_{(q_i \in Q)} IDF(q_i) \times \frac{f(q_i, D)(k_1 + 1)}{f(q_i, D) + k_1 \left(1 - b + \frac{b|D|}{avgdl}\right)}$$

در این فرمول:

- D نشان دهنده سند مورد بررسی و Q نشان دهنده پرسش کاربر است.
- q_i یکی از واژه های موجود در پرسش است.
- $f(q_i, D)$ تعداد تکرار واژه q_i در سند D است.
- $|D|$ طول سند بر اساس تعداد کلمات یا توکن ها است.
- $avgdl$ میانگین طول اسناد در مجموعه داده است.
- k_1 و b پارامترهایی هستند که نحوه تأثیر تعداد تکرار واژه و طول سند را کنترل می کنند. معمولاً مقادیر رایج برای آنها به ترتیب حدود (۱.۲) تا (۲) و (۰.۷۵) هستند.

بخش IDF اهمیت یک واژه را بر اساس میزان حضور آن در کل مجموعه اسناد مشخص می کند. واژه هایی که در تعداد زیادی از اسناد دیده می شوند، اطلاعات کمتری درباره موضوع سند ارائه می دهند و وزن کمتری دریافت می کنند. در مقابل، واژه هایی که فقط در تعداد محدودی از اسناد وجود دارند، معمولاً قدرت تشخیص بیشتری دارند و وزن بالاتری دریافت می کنند.

فرمول مورد استفاده برای IDF در BM25 به صورت زیر است:

$$IDF(q_i) = \ln \left(1 + \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} \right)$$

که در آن:

- N تعداد کل اسناد موجود در مجموعه است.
 - $n(q_i)$ تعداد اسنادی است که واژه q_i در آن‌ها مشاهده شده است.
- یکی از مزیت‌های اصلی BM25، عملکرد مناسب آن در جستجوی دقیق واژه‌ها و اصطلاحات است. این ویژگی در متون حقوقی اهمیت دارد؛ زیرا شماره ماده، نام قانون و بسیاری از اصطلاحات حقوقی می‌توانند اطلاعات بسیار مشخصی درباره سند مورد نظر ارائه کنند. با این حال، BM25 در تشخیص شباهت معنایی میان عبارت‌هایی که از واژه‌های متفاوتی برای بیان یک مفهوم استفاده می‌کنند، محدودیت دارد. به عنوان مثال در متن قانونی نوشته شده است «اصل اول قانون اساسی جمهوری اسلامی ایران» و اگر کاربری جستجو کند «اصل یک قانون اساسی جمهوری اسلامی ایران»، این جستجوگر امکان درک اینکه دو کلمه «یک» و «اول» ارتباط تنگاتنگی دارند را ندارد

۲-۳-۲. جستجوی معنایی و بردارهای تعبیه شده

برای برطرف کردن بخشی از محدودیت‌های جستجوی واژگانی که در بالا آمده است، می‌توان از جستجوی معنایی (Semantic Search) استفاده کرد. در این روش، متن به یک نمایش عددی با نام بردار تعبیه‌شده (Embedding) تبدیل می‌شود. این بردار ویژگی‌های معنایی متن را در فضای عددی نمایش می‌دهد که در بالا به آن اشاره شد.

در جستجوی معنایی، ابتدا پرسش کاربر به یک بردار تبدیل شده و سپس میزان شباهت آن با بردارهای مربوط به اسناد محاسبه می‌شود. در نتیجه، اسنادی که از نظر معنایی به پرسش نزدیک‌تر هستند (که یعنی فاصله عددی بردارها نسبت به هم تا حد قابل قبولی کم هستند) می‌توانند حتی در صورت نداشتن کلمات کاملاً یکسان، به عنوان نتایج مرتبط بازیابی شوند.

یکی از معیارهای متداول برای این کار، شباهت کسینوسی (cosine similarity) است که شباهت میان دو بردار را بر اساس زاویه میان آنها مشخص می‌کند (بردارهایی که در این حوزه استفاده می‌شوند، بیشتر از سه بعد دارند و تصور کردن آنها ممکن نیست). فرمول کلی آن به صورت زیر است:

$$\text{CosineSimilarity}(A, B) = \frac{A \cdot B}{|A||B|}$$

که در آن عبارت صورت ضرب داخلی بردار هاست و عبارت مخرج ضرب اندازه بردارها با هم است (منظور اندازه A با اندازه B). در این فرمول، هر چه عدد بدست آمده به یک نزدیک تر باشد، یعنی فاصله این دو بردار از نظر جهت به همدیگر نزدیک است و معنای آنها مشابه است.

از چالش های این قسمت وابستگی به مدل embedding است. در حوزه تبدیل کلمات به embedding کار های فراوانی انجام شده است که کلمات با معانی شبیه به هم کنار یکدیگر قرار بگیرند، ولی اگر مدلی که کلمات را به عدد تبدیل می کند، اگر دقت لازم را نداشته باشد، نتایج می تواند کاملاً نامرتب با پرسش کاربر باشد. چالش دیگر آن این است که این نوع جستجو کلمات را دقیقاً نگاه نمی کند؛ به عنوان مثال، اگر کاربر به دنبال «اصل یک قانون اساسی» باشد، چون این جستجوگر صرفاً با توجه به معانی کلمات اقدام به جستجو و بازیابی اطلاعات می کند، ممکن است «اصل چهار قانون اساسی» را نیز در نتیجه خود بیاورد. همچنین اگر متن بسیار طولانی شود، بار محاسباتی و دقت این جستجوگر افت محسوس پیدا می کند.

۳-۳-۲. بازرتبه بندی نتایج

پس از مرحله بازیابی اولیه، ممکن است تعداد نسبتاً زیادی نتیجه در اختیار سامانه قرار گیرد (نتایج، مجموع نتایج از هر دو جستجوگر است). با وجود آنکه این نتایج توسط روش های بازیابی انتخاب شده اند، همه آنها الزاماً به یک اندازه با پرسش کاربر مرتبط نیستند. بنابراین، مرحله دیگری برای ارزیابی دقیق تر ارتباط میان پرسش و نتایج بازیابی شده مورد نیاز است.

بازرتبه بندی (Reranking) فرایندی است که در آن مجموعه ای از نتایج اولیه با استفاده از یک مدل دقیق تر دوباره ارزیابی و مرتب می شوند. در این مرحله، پرسش کاربر و متن هر نتیجه به صورت یک زوج در اختیار مدل بازرتبه بند قرار می گیرند و مدل میزان ارتباط آنها را تعیین می کند.

مدل reranker یک شبکه عصبی cross-encoder است که طی فرایندی که کمی بالاتر توضیح داده شد، به هر نتیجه یک امتیاز ارتباط می دهد. توجه شود که این مدل نیز مانند باقی مدل های مولد است، با این تفاوت که تعداد پارامتر های آن در مقایسه با مدل های مولد بسیار کمتر است و دلیل این امر این است که مدل تنها اقدام به امتیازدهی در مورد مرتبط بودن سوال و پاسخ می کند و متن دیگری تولید نمی کند.

به عنوان مثال اگر برای پرسش «اصل یک قانون اساسی» نتایجی مانند زیر توسط جستجوگر های دیگر ارائه شود:

- «لایحه همه پرسى قانون اساسى جمهورى اسلامى ایران»
- «قانون اساسى جمهورى اسلامى ایران»
- «قانون تفسیر اصل ۳۷ متمم قانون اساسی»
- «منشور حقوق شهروندی مصوب رئیس جمهور»

مدل همه پاسخ های بدست آمده را بررسی می کند و ارتباطشان با سوال پرسیده شده را بررسی می کند. سپس به هر پاسخ یک امتیاز می دهد. دقت شود که در این مرحله الزاماً بردار ها مقایسه نمی شوند.

۴-۲. تولید تقویت شده با بازیابی

مدل های زبانی بزرگ توانایی بالایی در تولید متن و پاسخ گویی به پرسش ها دارند، اما دانش موجود در پارامتر های آن ها محدود به داده هایی است که در فرایند آموزش مدل مورد استفاده قرار گرفته است. بنابراین، مدل لزوماً به آخرین نسخه اطلاعات، اسناد خاص یا منابعی که باید برای یک پاسخ مشخص مورد استفاده قرار گیرند، دسترسی ندارد. علاوه بر این، در صورتی که اطلاعات کافی برای پاسخ گویی در اختیار مدل نباشد، ممکن است متنی تولید کند که از نظر زبانی و ظاهری صحیح به نظر برسد، اما از نظر صحیح بودن قابل اعتماد نباشد و توهّم زده باشد. این مسئله در کاربردهای تخصصی، به خصوص در حوزه حقوق، اهمیت بیشتری پیدا می کند.

تولید تقویت شده با بازیابی (Retrieval-Augmented Generation یا RAG) (بله، این ترجمه آن است) روشی است که با ترکیب بازیابی اطلاعات و تولید متن توسط مدل زبانی، بخشی از این محدودیت ها را کاهش می دهد. ایده اصلی RAG این است که مدل زبانی به جای آنکه تنها بر دانش ذخیره شده در پارامترهای خود تکیه کند، هنگام پاسخ گویی به اطلاعات مرتبطی که از یک منبع خارجی بازیابی شده اند نیز دسترسی داشته باشد یا حوزه دانشی خود را صرفاً محدود کند به آنچه برای آن مورد استفاده قرار می گیرد.

فرایند کلی RAG را می توان به سه مرحله اصلی تقسیم کرد. ابتدا پرسش کاربر دریافت و برای پیدا کردن اطلاعات مرتبط در مجموعه داده استفاده می شود. سپس اسناد یا بخش هایی از اسناد که بیشترین ارتباط را با پرسش دارند بازیابی می شوند. در مرحله آخر، اطلاعات بازیابی شده به همراه پرسش کاربر در اختیار مدل زبانی قرار می گیرند و مدل پاسخ نهایی را بر اساس این اطلاعات تولید می کند.

۵-۲. تعاریف کوتاه دیگر

۱-۵-۲. شاخص بندی

داده های ذخیره شده در پایگاه داده شاید همه آن اطلاعاتی که مورد نیازمان است را فراهم داشته باشند و جستجوگر مشکلی در پیدا کردن اطلاعات لازم نداشته باشد، ولی در بسیاری از حوزه ها (از جمله همین حوزه)، حجم اطلاعات ذخیره شده در پایگاه داده بسیار زیاد است به طوری که سرعت جستجو را بسیار کند کند و زمان پاسخ گویی را بسیار بالا ببرد.

شاخص بندی (Indexing) فرایندی است که طی آن داده ها و اسناد به شکلی سازمان دهی و آماده می شوند که بتوان در زمان کوتاه تری، اطلاعات مورد را با سرعت بیشتری پیدا کرد. شاخص بندی با ایجاد ساختار های از پیش آماده، این نیاز را کاهش می دهد و امکان جست و جوی سریع تر را فراهم می کند. از آنجایی که بحث مربوط به indexing بسیار گسترده است، وارد جزئیات پیچیده آن نمی شویم.

۲-۵-۲. Django

جنگو (Django) یک قالب (یا framework) مخصوص زبان پایتون است که در آن می توان یک برنامه تحت وب را بالا آورد و قسمت های مختلف سامانه را (دیتابیس به برنامه و برنامه به رابط کاربری) به راحتی به هم متصل کرد. Django یک زبان برنامه نویسی مستقل نیست، بلکه محیطی فراهم می کند که کدزنی و اتصال بخش های مختلف وبسایت پیچیدگی کمتری داشته باشد.

۲-۵-۲. Postgres

Postgres یک پایگاه داده متن باز (open-source) است که برای ذخیره سازی داده های ساختاریافته استفاده می شود. در این پایگاه داده اطلاعات در قالب جدول هایی شامل سطر و ستون ذخیره می شود و ارتباط میان داده های مختلف با استفاده از کلید ها و روابط میان جدول ها مشخص می شود.

۲-۵-۲. Milvus

Milvus یک پایگاه داده متن باز (open-source) است برای ذخیره سازی بردارهای تعبیه شده (Embedding) طراحی شده است. این پایگاه داده برخلاف پایگاه داده بالا، داده ها را بدون scheme یا طرح ذخیره می کند (در اصل بردارها داده های ذخیره شده در این پایگاه داده هستند). این پایگاه داده در حوزه هوش مصنوعی یکی از قوی ترین انتخاب هاست.

۲-۵-۲. RRF (Reciprocal Rank Fusion)

وقتی نتیجه چند جستجو را دریافت کردیم، باید آنها را با توجه به مقیاس امتیاز دهی ای که خود جستجوگر استفاده کرده آنها را در یک لیست ادغام کنیم و صرفاً جمع کردن امتیاز هر کدام از داده ها (بر اساس score هر مورد یافته شده توسط هر جستجوگر) کافی نیست (امتیازدهی BM25 مقیاس بزرگتری در مقایسه با حالت جستجوی معنایی دارد).

RRF یک الگوریتم برای ترکیب چند فهرست رتبه بندی شده از نتایج جستجو است که امتیاز نهایی با توجه به رتبه نتیجه در فهرست های مختلف محاسبه می شود. به صورت کلی فرمول آن به صورت زیر است:

$$RRF(d) = \sum_{r \in R} \frac{1}{k + rank_r(d)}$$

در این رابطه، d نتیجه مورد نظر، R مجموعه روش های رتبه بندی، $rank_r(d)$ رتبه نتیجه d در روش r و k یک ثابت برای کنترل میزان تأثیر رتبه است. هرچه رتبه یک نتیجه در یک روش بالاتر باشد، سهم آن روش در امتیاز نهایی بیشتر خواهد بود.

۳. فصل سوم: طراحی سامانه

۳-۱. مقدمه کوتاه

معماری پایه ای برنامه به صورت یک برنامه یکپارچه و ماژولار بر بستر چارچوب django طراحی و پیاده سازی شده است. در این معماری بخش های مختلف سامانه در قالب لایه ها و اجزای دارای وظیفه مشخص سازمان دهی شده اند. به لطف این تفکیک وابستگی بین لایه های مختلف به شدت کاهش پیدا می کند و تست کردن بخش های مختلف وبسایت آسان می شود.

این سامانه از لایه های زیر تشکیل شده است:

- Django (پایه ای ترین لایه سامانه)
- داده
- بازیابی اطلاعات
- RAG
- رابط کاربری

لایه Django قسمتی نیست که توسط ما توسعه یافته باشد، پس این لایه را توضیح نمی دهیم. جریان کلی سامانه را می توان به صورت زیر خلاصه کرد:

لایه	مسئولیت	اجزای اصلی
داده	ذخیره پایدار و روابط حقوقی	Django، مدل های PostgreSQL
آماده سازی	نرمال سازی، طبقه بندی و تبدیل JSON به ساختار حقوقی	Hazm، Importer، دستورات مدیریتی
بازیابی	یافتن مواد مرتبط با پرسش	Persian Embeddings، BM۲۵، Milvus، RRF
دقت نهایی	رتبه بندی مجدد کاندیداها	safora/reranker-xlm-roberta-large
تولید پاسخ	ساخت زمینه و پاسخ JSON کنترل شده	RAG، OpenRouter/OpenAI-compatible client
روابط حقوقی	استخراج ارجاع و اثر اصلاح/نسخه/الحاق	ReferenceExtractor، ActionExtractor

جدول ۱ - لایه های سیستم

۲-۳. قسمت استخراج اطلاعات

این لایه در ساختار لایه بندی شده بالا قرار نگرفته است چون اجرای این قسمت از برنامه کاملاً مستقل از **django** است و بدون نیاز به آن نیز کار می کند. این قسمت مسئول استخراج متون حقوقی از سایت های قانونی موجود در اینترنت است که در چندین مرحله انجام شده است:

در مرحله اول، چند جستجو برای یافتن سایت های حقوقی انجام شده است که سایت های روزنامه رسمی جمهوری اسلامی ایران به نشانی **rrk.ir**، سامانه ملی قوانین و مقررات جمهوری اسلامی ایران به نشانی **qavanin.ir**، و سایت نظامات به نشانی **nezamat.ir** یافت شدند. از بین این سایت ها، سایت نظامات برای استخراج اطلاعات انتخاب شد و این انتخاب چند دلیل دارد:

۱. استخراج از منابع دیگر دردسر های بسیار فراوانی داشت. سایت **qavanin.ir** گاهی نتیجه های تکراری به کاربر نشان می داد و گاهی نیز به خاطر زیرساخت ضعیفش به **error** های غیر قابل رفع برخورد می شد (این ارور ها از جنس سرور بودند تا کاربر). سایت **rrk.ir** نیاز به ساخت یک **session** و ایدی یکسان بود که استخراج را دشوار می کرد

۲. بسیاری از مصوبات، بخشنامه ها و به طور کلی، رکورد های دو سایت دیگر به قوانین خاص یک استان اشاره می کرد تا قوانین کل کشور. فیلتر کردن این رکورد ها صرفاً با قرار دادن یک فیلتر لغوی مثل **regex** بر روی عنوان رکورد امکانپذیر نبود، چرا که برخی قوانین شاید به یک استان اشاره می کرد، ولی تصمیم آن می توانست در سطح کل کشور نیز تاثیر بگذارد؛ به عنوان مثال: «تصویب بودجه پیشنهادی رئیس جمهور در باب /حیای دریایچه/رومیه»

۳. سایت روزنامه رسمی متون هر مصوبه و قانون را در قالب یک فایل **pdf** در سامانه بارگذاری می کرد که استخراج اینگونه متون بسیار دشوار است.

۴. متون حقوقی در سایت **nezamat** در مقایسه با سایت های دیگر ساختاریافته تر از سایت های دیگر بود.

در مرحله دوم، یک ابزار **webdriver** اجرا می شود که لینک های هر سایت در یک فایل جداگانه ای ذخیره شود که در مرحله بعدی متن آنها استخراج شود. این داده ها در پوشه **links** قرار گرفته است. برای این کار از ابزار **selenium** استفاده شده است.

با توجه به ساختار قوانین در سایت، برای اعمال فیلتر اولیه قوانین، واژه اول عنوان سند حقوقی، قسمت دسته بندی و تاریخ تصویب هر متن حقوقی در نظر گرفته شده است که عنوان سند نباید با واژگانی مانند «نظر، رای، دادنامه، مذاکرات و ...» شروع شده باشد. لیست کامل این واژگان در کد مربوط به استخراج لینک ها قرار گرفته شده است. همچنین در قسمت دسته بندی قوانین، نباید عبارت «قانون خارجی» وجود داشته باشد (توجه شود که متون با عبارت «قرارداد خارجی» چون مربوط به ایران می شود، حذف نشده اند). تاریخ تصویب نیز نباید به تاریخ قبل از سال ۱۳۰۰ برگردد (علت این انتخاب این است که قانونی مانند قانون مدنی در تاریخ ۱۳۱۰ تصویب شده است و در طول زمان به دلیل مصوبات و اصلاحیه های ابلاغی تغییر یافته است).

در مرحله سوم، ابزار **webdriver** وارد لینک های استخراج شده می شود و متن را استخراج می کند و متن استخراجی را با یک الگوریتم خاص، ساختار بندی می کند. الگوریتم به صورت خلاصه اینگونه است:

۱. یک فایل **json** بساز
۲. متن را جستجو کن، عبارات ساختاری را در متن بیاب و آن عبارت ساختاری را که یافتی **pivot** قرار بده (عبارات ساختاری در فایل آمده است که متشکل است از بحث، کتاب، فصل، مبحث و ...)
۳. اگر **pivot** خالی نبود، متن را با توجه به آن تکه تکه کن؛ وگرنه برو به ۴
۴. برای هر تکه:
 - ۴/۱. الگوریتم را برای آن تکه صدا بزن
۵. متن را جستجو کن و عبارات مقررہ ای را در متن بیاب و آن عبارت مقررہ ای را که یافتی **subpivot** قرار بده (عبارت مقررہ ای به نام **provision** آمده است مثل ماده، تبصره، بند و ... یافتن بند با یافتن شماره بندی یا حرف بندی انجام می شود).
۶. اگر **subpivot** خالی نبود، متن را با توجه به آن تکه تکه کن، وگرنه تمام.
۷. برای هر تکه:
 - ۷/۱. الگوریتم را برای آن تکه صدا بزن
۸. تکه های بدست آمده را با توجه به لایه کنونی به ساختار **json** در بیاور و در فایل بنویس.
۹. تمام.

(متأسفانه به خاطر بعضی از حالات استثنا، مجبور شدیم که قسمت **children** را پاک کنیم و همه **json** ها را یک سطح در نظر بگیریم؛ چون نسخه اولیه فایل الگوریتم من بین عبارت ساختاری و زیرساختاری فرقی قائل نبود و آن هم به خاطر چند مورد استثنای دیگر بود).

نکته مهم: متأسفانه اکنون این قسمت قابلیت بازتولید خود را از دست داده است، چون سازنده سایت نظامات یک سامانه **anti-bot** در سایت خود راه اندازی کرده است که تعامل کاربر و **webdriver** نبودن را از شروط اجازه دسترسی به سایت قرار داده است، اما داده های استخراج شده در پوشه **data** در فرمت **rar** ذخیره سازی شده است. رکورد های ذخیره شده تا پنجم شهریور سال ۱۴۰۵ بروزرسانی شده است.

۳-۳. معماری قسمت پایگاه داده

در لایه داده از دو پایگاه داده **Postgres** و **Milvus** استفاده شده است. **Postgres** به عنوان پایگاه داده اصلی برای نگهداری اطلاعات ساختار یافته و **Milvus** برای ذخیره بردار های **Embedding** برای جستجوی معنایی به کار می رود. **Postgres** به این دلیل استفاده شده است که داده ها به اندازه قابل قبولی ساختار یافته هستند و **Milvus** برای جستجوی معنایی استفاده می شود. می توانستیم برای بردار ها از ویژگی **pgvector** در **postgres** استفاده کنیم، ولی این مورد تنها در پروژه های سبک و ساده استفاده می شود و برخلاف

Milvus مقیاس پذیر نیست. متن در Postgres در مدل های زیر ذخیره می شود (مقصود از موجودیت همان entity در پایگاه داده است):

در بالاترین سطح، LegalDocument برای نمایش یک سند حقوقی در نظر گرفته شده است. این موجودیت اطلاعاتی مانند عنوان سند، نوع سند، مرجع صادرکننده، تاریخ تصویب، تاریخ انتشار، تاریخ اجرا و وضعیت سند را نگهداری می کند. هر سند حقوقی می تواند شامل مجموعه ای از عناصر ساختاری و مقررات باشد. این موجودیت سلسله مراتب قوانین را نیز نگهداری می کند؛ به عنوان مثال قانون اساسی بر هر قانونی در کشور حاکم است.

برای حفظ ساختار داخلی اسناد، LegalElement طراحی شده است. این موجودیت امکان ایجاد ارتباط سلسله مراتبی میان عناصر مختلف یک سند را فراهم می کند. عناصری مانند کتاب، بخش، فصل، مبحث و سایر بخش های ساختاری سند از طریق این ساختار قابل نمایش هستند. در نتیجه، جایگاه هر قسمت از یک سند در ساختار کلی آن قابل نگهداری است. توجه شود که این موجودیت، صرفاً ساختار را نگه می دارد و متن قانونی در آن قرار نمی گیرد.

در سطح پایین تر، LegalProvision برای نمایش مقررات حقوقی استفاده می شود. هر مقررہ دارای اطلاعاتی مانند نوع مقررہ، شماره، عنوان و متن است و از طریق عنصر ساختاری مربوطه به سند حقوقی متصل می شود. این تفکیک باعث می شود واحد اصلی مورد استفاده در فرایند بازایی، یعنی متن یک مقررہ، به صورت مستقل قابل شناسایی و بازایی باشد. منظور از مقررات همان ماده ها و تبصره ها هستند.

علاوه بر متن اصلی مقررات، ساختار داده سامانه امکان نگهداری نسخه های مختلف یک مقررہ را نیز فراهم می کند. LegalVersion برای ثبت نسخه های مختلف متن یک مقررہ و وضعیت هر نسخه مورد استفاده قرار می گیرد. این موضوع در متون حقوقی اهمیت زیادی دارد، زیرا یک مقررہ ممکن است در طول زمان اصلاح یا با نسخه جدیدی جایگزین شده باشد و پاسخ به یک پرسش حقوقی باید تا حد امکان بر اساس نسخه معتبر و مرتبط با زمان مورد نظر کاربر ارائه شود.

همچنین برای نمایش ارتباط میان مقررات، موجودیت LegalRelationship در نظر گرفته شده است. این موجودیت امکان ثبت روابطی مانند اصلاح، نسخ، جایگزینی، ارجاع و سایر روابط میان مقررات را فراهم می کند. اطلاعات تکمیلی مربوط به هر رابطه نیز در موجودیت RelationshipContext نگهداری می شود. این ساختار در طراحی فعلی سامانه به عنوان زیرساختی برای توسعه قابلیت های پیشرفته تر تحلیل روابط حقوقی در نظر گرفته شده است.

برای نگهداری اطلاعات مربوط به منبع اسناد نیز SourceDocument و DocumentSource در نظر گرفته شده اند. SourceDocument اطلاعات مربوط به یک نسخه یا نمونه دریافت شده از یک سند از منبع را نگهداری می کند. این موجودیت باعث می شود اطلاعات مربوط به خود سند حقوقی از اطلاعات مربوط به منبعی که سند از آن دریافت شده جدا باشد.

DocumentSource برای ایجاد ارتباط میان یک سند حقوقی و منبع آن استفاده می شود. از طریق این موجودیت می توان مشخص کرد که یک **LegalDocument** از چه منبع یا منابعی دریافت شده است. این تفکیک به خصوص زمانی اهمیت دارد که یک سند حقوقی از چند منبع مختلف قابل دریافت باشد یا اطلاعات یک سند در طول زمان از منبع مشخصی استخراج و به روزرسانی شود. در نتیجه، اطلاعات مربوط به محتوای حقوقی **LegalDocument** نگهداری می شود، در حالی که اطلاعات مربوط به منشأ و ارتباط آن با منابع در موجودیت های مربوط به **SourceDocument** و **DocumentSource** قرار می گیرد.

برای استخراج داده ها، چندین سرویس قرار داده شده است:

- **Importer**: این فایل اصلی ترین قسمت استخراج متون قانونی است که مدل های بالا را ساخته و در کلاس خود قرار می دهد. الگوریتمی که اینجا برای بدست آوردن ارتباطات پدر فرزندی استفاده شده (منظور اینکه اگر یک ماده ای تبصره دارد، تبصره در متغیر **parent** خود این ماده را داشته باشد) به شرح زیر است:

۱. یک استک خالی بساز.

۲. **Json** را با توجه به ساختارش به قسمت های مختلف تقسیم کن.

۳. برای هر قسمت:

۳/۱

۳/۱/۱. اگر نوع مقرر (ماده، تبصره، فصل، کتاب، ...) در بالای استک بود، هیچ کاری نکن

۳/۱/۲. وگرنه از استک بردار تا زمانی که سر استک هم وزن نوع مقرر ای باشد که اکنون در

حال پردازش هستی، یا استک خالی شود.

۳/۲. کلاس را بساز و پدرش را آن مقرر ای قرار بده که یکی پایین تر است (یعنی سر استک نه،

زیر آن) (اگر استک فقط یک عنصر دارد، پس یعنی پدر ندارد)

- **Actions** و **References**: این قسمت مربوط به استخراج ارتباطات بین مواد مختلف است. قسمت

Actions به نوع ارتباط اشاره می کند؛ به عنوان مثال: اصلاحیه، نسخ، اضافات، ... **References** به

پیدا کردن قسمت متن که در حال اشاره کردن به متن حقوقی دیگر است؛ به عنوان مثال متنی مثل «به

استناد به اصل ۱۲۷ قانون اساسی»

موجودیت های سامانه در جدول زیر خلاصه شده اند:

موجودیت	کاربرد در سامانه
LegalEntity	هویت یکتای سند یا موجودیت قانونی در سطح مفهومی
LegalDocument	فراداده قانون: عنوان، نوع، مرجع، تاریخ، سلسله مراتب و وضعیت
LegalElement StructuralElement	نگهداری درخت ساختارهایی مانند کتاب، بخش، فصل و قسمت
LegalProvision	واحد قابل استناد: اصل، ماده، تبصره، بند، جزء یا مورد دیگر

متن و وضعیت نسخه‌های زمانی یک مقرر	LegalVersion
روابط بین مواد و شواهد مربوط به آن‌ها	LegalRelationship RelationshipContext
نگهداری اطلاعات مربوط به سند دریافت شده از یک منبع، مستقل از ساختار حقوقی اصلی آن	SourceDocument
ارتباط میان سند حقوقی و منبع آن و مشخص کردن منشأ اطلاعات سند	SourceDocument

جدول ۲ - مدل هایی که قوانین فارسی در آنها ذخیره می شود.

نکته مهم: متأسفانه به دلیل ضعف سخت افزاری و محدودیت زمانی امکان به استخراج روابط بین ماده ها نبودم.

۳-۴. معماری قسمت بازیابی اطلاعات

یک یادآوری کوچک: مقصود از بازیابی retrieval است و نه recovery

فرایند بازیابی از پردازش و نرمال سازی پرسش آغاز می‌شود. در این مرحله، پرسش کاربر پس از بررسی اولیه و حذف فضاهای غیرضروری، با استفاده از ابزارهای پردازش زبان طبیعی نرمال سازی می‌شود تا اختلاف های نوشتاری موجود در متن فارسی، مانند تفاوت در فاصله گذاری و شکل برخی نویسه‌ها، تأثیر کمتری بر فرایند بازیابی داشته باشد. برای این قسمت کتابخانه hazm را مورد استفاده قرار دادیم، چون در مقایسه با مدل های دیگر مانند parsivar نتیجه بهتری را ارائه می داد.

پس از پردازش پرسش، دو مسیر بازیابی اجرا می‌شوند. مسیر نخست از روش BM25 برای بازیابی واژگانی استفاده می کند. دلیلش پیدا کردن نمونه هایی است که متنی به پرسش کاربر بسیار نزدیک هستند (تنها از نظر لغوی و نه از نظر معنایی).

در مسیر دوم، پرسش به یک بردار عددی یا embedding تبدیل می‌شود. با محاسبه شباهت میان embedding پرسش و embedding مقررات، مجموعه ای از مقررات دارای ارتباط معنایی با پرسش بازیابی می شوند.

برای اینکه نتیجه پرسش ها (در هر دو جستجوگر) کیفیت لازم را داشته باشد، جستجو به این صورت انجام می شود: ابتدا برای هر مقرر یک گروه ساخته می شود که شامل همه زیرمقرر هایش (مانند تبصره های یک ماده) و مسیرش تا سند قانونی و عنوان سند است. جستجو در این گروه ها انجام می شود. پایگاه داده Milvus باید این گروه ها را درون خود Index کند تا قابل بازیابی باشد. برای مدلی که embedding را انجام دهد، از Tooka-Sbert-V2-Large استفاده شده است که محصول شرکت پارت است.

نتایج حاصل از دو روش BM25 و بازیابی معنایی سپس با استفاده از روش RRF با یکدیگر ترکیب می شوند. در این قسمت به هر دو جستجو گر وزن یکسان داده می شود، چون یافتن شماره مقرر همانقدر اهمیت دارد که یافتن سند و مقرر ای که معنایش با پرسش همخوانی دارد.

پس از ترکیب نتایج، تعدادی از مقررات برتر به مرحله بازرتبه بندی ارسال می شوند. در این مرحله، مدل Reranker هر جفت شامل پرسش و متن مقرر را به صورت مستقیم بررسی کرده و میزان ارتباط آن ها را ارزیابی می کند. برخلاف مرحله بازیابی اولیه که برای جستجو در حجم زیادی از داده طراحی شده است، بازرتبه بندی تنها روی تعداد محدودی از نامزدهای بازیابی شده انجام می شود و بنابراین می تواند تحلیل دقیق تری از ارتباط پرسش و مقرر ارائه دهد.

نسخه اولیه پروژه از مدل محلی safora/XLM-RoBERTa-Large استفاده می کند که بر روی متون علمی آموزش داده شده است. نسخه فعلی از cohere-rerank-v4.0-pro استفاده می کند که برخلاف مدل اول که منحصر بر روی متون فارسی آموزش داده شده است، چند زبانه است.

برای جلوگیری از ورود نتایج ضعیف به مرحله تولید پاسخ، در مرحله بازرتبه بندی دو شرط در نظر گرفته شده است. ابتدا، در صورتی که امتیاز بهترین نتیجه از یک آستانه مشخص کمتر باشد، نتیجه ای به عنوان پاسخ مناسب به مرحله بعد ارسال نمی شود. همچنین نتایجی که اختلاف زیادی با بهترین نتیجه داشته باشند حذف می شوند. بنابراین تعداد نتایج نهایی ثابت نیست و بر اساس میزان اطمینان حاصل شده از ارتباط نتایج تعیین می شود.

در نهایت، نتایج تأیید شده توسط این فرایند به بخش تولید پاسخ منتقل می شوند و به عنوان مبنای ایجاد زمینه حقوقی مورد استفاده قرار می گیرند.

جدول زیر روش های مختلف بازیابی داده را نشان می دهد:

روش	مزیت	ضعف در صورت استفاده تنها	نقش در طراحی
BM25	دقت بالا برای واژگان و شماره های دقیق	درک ضعیف تر از بازیابی معنایی	پوشش تطابق لفظی
برداری	پوشش مشابهت معنایی و عبارت های متفاوت	ممکن است ماده ظاهراً مشابه اما نادرست بیاورد	پوشش مفهوم
RRF	ترکیب پایدار دو رتبه بندی	خودش ارتباط عمیق را نمی سنجد	ساخت فهرست کاندیدا
Cross-Encoder	قضاوت دقیق پرسش در برابر متن ماده	هزینه محاسباتی بالاتر	فیلتر و رتبه بندی نهایی

جدول 3 - روش های جستجو در پایگاه داده

در تمامی قسمت های بازیابی اطلاعات و دادن نتایج به RAG یک فایل log ساخته می شود که در لحظه بتوان متن پردازش شده و نتایج یافته شده توسط هر جستجوگر را بررسی کرد و از سالم بودن الگوریتم ها اطمینان حاصل نمود.

۴. فصل چہارم: اجرا

۱-۴. اسختراج داده

در مرحلهٔ نخست کار استخراج قوانین و مقررات انجام می شود. فایل های مربوط به استخراج قوانین به شکل فایل `jupyter` در پوشهٔ `Data` قرار گرفته شده است. ابتدا باید فایل `nezamat_link_extraction` اجرا شود تا لینک های مربوط به سایت `nezamat.ir` استخراج شود. پس از اجرا باید حدود ۴۰ هزار لینک در فایل به نام `nezamat.txt` قرار می گیرد.

```
title---دیوان عدالت اداری: ابطال نامه مورخ 1404/4/9 مدیرکل دفتر توسعه مهندسی ساختمان وزارت راه و شهرسازی  
url---https://nezamat.ir/%d8%b1%d8%a3%db%8c-%d8%b4%d9%85%d8%a7%d8%b1%d9%87-%db%b5%db%b4%db%b7%db%b3%db%b0%  
excerpt---  
approval_date---۱۴۰۵/۰۳/۰۵  
number---  
publication_date---۱۴۰۵/۰۶/۰۱  
category---هیأت عمومی دیوان عدالت اداری  
category_url---https://nezamat.ir/%d9%85%d9%82%d8%a7%d9%85%d8%a7%d8%aa/%d8%af%db%8c%d9%88%d8%a7%d9%86-%d8%b  
source---nezamat.ir  
  
title---رأی شماره ۵۴۷۱۵۸ - ۱۴۰۵/۰۳/۰۵ هیئت عمومی دیوان عدالت اداری: اعلام تعارض در آرای شعب دیوان عدالت اداری  
url---https://nezamat.ir/%d8%b1%d8%a3%db%8c-%d8%b4%d9%85%d8%a7%d8%b1%d9%87-%db%b5%db%b4%db%b7%db%b1%db%b5%  
excerpt---  
approval_date---۱۴۰۵/۰۳/۰۵  
number---  
publication_date---۱۴۰۵/۰۶/۰۱
```

شکل ۱- نمونه لپینک های استخراج شده

اکنون که لینک ها استخراج شده اند، مرحله بعدی استخراج ماده ها، تبصره ها، اصل ها و ... است. در این مرحله، فایل `nezamat_json_exrtaction` باید اجرا شود که لینک ها را یک به یک باز کند و به متن حقوقی ساختار بدهد. پس از اجرای این فایل، باید فایل `nezamat_json_repair` باید اجرا شود. این فایل مشکل الگوریتمی که برای بخش استخراج ساختارمند متن حقوقی به کار رفته بود از بین می رود (مشکل اینجا حل نمی شود بلکه در قسمت ورود به پایگاه داده حل می شود). این یک نمونه از ساختار متن حقوقی است:

```

17 "type": "introduction",
18 "marker": null,
19 "header": " ",
20 "text": "۱۴ با اصلاحاتی در عنوان و متن به شرح زیر به تصویب رسید\n\ngزارش کمیسیون امنیت ملی و سیاست خارجی به مجلس شورای اسلامی",
21 "children": []
22 },
23 {
24 "type": "structural",
25 "marker": "فصل",
26 "header": "فصل اول - کلیات و تعاریف",
27 "text": " ",
28 "children": []
29 },
30 {
31 "type": "provisions",
32 "marker": "ماده",
33 "header": " ",
34 "text": " ",
35 "children": []
36 },
37 {
38 "type": "provision",
39 "marker": "ماده",
40 "header": "- ماده ۱",
41 "text": "۱. اجتماعی یا فرهنگی شود ممنوع است. متخلفان از حکم این ماده حسب مورد، مشمول مجازاتهای مقرر در این قانون خواهند بود.",
42 "children": []

```

شکل ۲- نمونه ساختار استخراج شده از سند قانونی

نسخه اولیه فایل های json که شامل ساختار این ماده ها و این بند هاست، در پوشه `data/json` و نسخه بدون مشکل آنها در پوشه `data/valid_json` قرار گرفته است. پس از اجرای

نکته مهم: همانطور که بالاتر ذکر شد، این قسمت اکنون قابلیت بازتولید خود را از دست داده است. بنابراین باید وارد مرحله ۲-۴ شویم (در مخزن پروژه، فایل فشرده این فایل ها قرار گرفته شده است و نیاز به استخراج دارد).

نکته مهم: فراموش شده بود که اصل نیز باید مانند ماده ساختار بندی شود. بنابراین در قسمت آماده سازی زیرساخت به این مشکل رسیدگی می شود.

۲-۴. آماده سازی زیرساخت

اکنون که داده ها استخراج شده اند، باید زیرساخت مربوط به ورود متون حقوقی به پایگاه داده ساخته شود. برای این کار یک فایل `docker-compose` در پروژه قرار داده شده است که به کمک دستور زیر، پایگاه ها آماده کار می شوند:

docker compose up -d

در انتخاب نسخه های ابزار های مورد نیاز، نسخه ای انتخاب شده است که آخرین ورژن نسل قبلی باشد (یعنی اگر به عنوان مثال `milvus` ورژن 3.x آخرین نسخه است، آخرین نسخه سری 2.x انتخاب شده است).

۳-۴. انتقال و آماده سازی داده استخراج شده

اکنون ما قوانین را به صورت ساختارمند در اختیار داریم. ابتدا باید `django` را از پایگاه داده ای که ساخته شده مطلع سازیم. برای این کار ابتدا دستور زیر را اجرا می کنیم:

python manage.py migrate

```
Applying legal.0003_legaldocument_hierarchy_level... OK
Applying legal.0004_alter_sourcedocument_source... OK
Applying legal.0005_alter_legaldocument_hierarchy_level... OK
Applying legal.0006_legaldocument_document_id... OK
Applying legal.0007_alter_legaldocument_document_id... OK
Applying legal.0008_alter_legaldocument_document_id... OK
Applying legal.0009_alter_legaldocument_document_id... OK
Applying legal.0010_rename_valid_from_legalversion_version_date_and_more... OK
Applying legal.0011_remove_legalversion_source... OK
Applying legal.0012_alter_documentsource_id_alter_legaldocument_id_and_more... OK
Applying legal.0013_alter_legalprovision_provision_type... OK
Applying legal.0014_alter_documentsource_id_alter_legaldocument_id_and_more... OK
Applying sessions.0001_initial... OK
```

شکل ۳- آماده سازی پایگاه های داده در `django`

در مرحله بعد به کمک دستور زیر متون قانونی را وارد پایگاه داده `postgres` می کنیم:

python manage.py import_json_documents <json_files>

```
[104/20796] 10123.json
    Imported: 1375 بومیم ء ادهش نیدل او یرم تسم شیزفا نوناق حالصا نوناق
[105/20796] 10124.json
    Imported: روشک یتیبرت و ینیم ات تام ادقا و اهان دنز نامزاس همان نیی آ هیحالصا
[106/20796] 10125.json
    Imported: دم آردمک نارچ اتسم هب همنحلافیرق تالی همت تخ ادرب هون همان نیی آ
[107/20796] 10126.json
    Imported: یاه راجا نکسم هفرع و شادح همان نیی آ
[108/20796] 10127.json
    Imported: روشک لک 1384 لاس هچدوب نوناق (2) همریت (ح) دنب یی راجا همان نیی آ حالصا
[109/20796] 10128.json
    Imported: [1384 بومیم] ییاضق لچس همان نیی آ
[110/20796] 10129.json
```

شکل ۴- ورود داده ها به postgres

پس از وارد کردن متون قانونی، باید یک قسمت از پایگاه داده را تصحیح کنیم: قانون اساسی به صورت اصل های جدا از هم در نیامده است و باید این مورد خاص رفع شود. بنابراین بعد از وارد کردن اطلاعات به داخل postgres باید شی مربوط به سند حقوقی قانون اساسی را حذف کنیم و دوباره آن را وارد ساختمان داده کنیم. برای این کار دستور زیر باید اجرا شود:

```
python manage.py fix_constitution --input <input_html> --document-id <doc_id>
```

که doc_id به شماره سند قانونی قانون اساسی در پایگاه داده اشاره دارد که در تصویر زیر توانستیم آن را استخراج کنیم:

```
>>> from legal.models import LegalDocument, HierarchyLevel
>>>
>>> LegalDocument.objects.filter(
...     hierarchy_level=HierarchyLevel.CONSTITUTION
... ).values("id", "title", "document_type", "hierarchy_level")
<QuerySet [{id': 5606, 'title': 'قانون اساسی جمهوری اسلامی ایران', 'document_type': 'law', 'hierarchy_level': 'constitution'}]>
>>>
```

شکل ۵- استخراج شماره سند از محیط shell در django

طبق عکس بالا، شماره سند برابر ۵۶۰۶ است.

اکنون داده مورد نیاز برای جستجوگر bm25 آماده است، اما ما یک جستجوگر دیگر برای milvus نیز داریم. پایگاه داده milvus را باید همانگونه که می خواهیم جستجو در آن انجام دهیم آماده کنیم (یعنی همان گروه بندی ای که در قسمت جستجو مد نظر داشتیم در این قسمت اعمال کنیم). برای این کار کد زیر را اجرا می کنیم. اجرای این بخش بسیار زمانبر خواهد بود (در سیستم من این کار حدود ۷ ساعت طول کشید):

```
python manage.py build_dense_index
```

۴-۴. اجرای نهایی

اکنون پایگاه داده آماده شده است و زمان اجرای قسمت رابط کاربری است. برای این کار باید دستورات زیر به ترتیب اجرا شود تا رابط کاربری react ساخته شود:

```
npm run build
```

```
npx serve -s dist
```

اکنون نوبت اجرای قالب اصلی، یعنی django است. دستور زیر این کار را برای ما انجام می دهد:

```
python manage.py runserver
```

```
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
September 13, 2026 - 21:31:41
Django version 5.2.17, using settings 'config.settings'
Starting development server at http://127.0.0.1:9876/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
```

شکل ۶- اجرای django

اکنون سرور ما آماده به کار است و برای بالا آوردن سایت، به نشانی ای که در اجرای npx به وجود آمده است برویم و در صفحه زیر سوال قانونی خود را بپرسیم.



شکل ۷- صفحه اول سایت

۵. فصل پنجم: جمع بندی

۵-۱. نتیجه گیری

در این پروژه، یک سامانه هوشمند برای پردازش، بازیابی و پاسخ‌گویی به پرسش‌های حقوقی به زبان فارسی طراحی و پیاده‌سازی شد. هدف اصلی پروژه، ایجاد سامانه‌ای بود که بتواند به جای اتکای صرف به دانش داخلی مدل زبانی، پاسخ‌های خود را بر اساس منابع و مقررات حقوقی موجود تولید کند.

برای دستیابی به این هدف، ابتدا ساختاری برای استخراج و سازمان‌دهی اسناد حقوقی طراحی شد و اطلاعات مربوط به اسناد، عناصر ساختاری، مقررات و نسخه‌های مختلف آنها در یک پایگاه داده ساختار یافته ذخیره شد. سپس برای بازیابی اطلاعات مرتبط با پرسش کاربر، از ترکیب روش‌های بازیابی لغوی و معنایی استفاده شد. روش BM25 برای تطابق دقیق واژه‌ها و عبارات و روش embedding برای تشخیص ارتباط معنایی میان پرسش و مقررات مورد استفاده قرار گرفت. نتایج این دو روش با استفاده از RRF ترکیب شده و سپس توسط مدل بازرتبه بندی ارزیابی شدند.

در مرحله تولید پاسخ نیز از معماری RAG استفاده شد. در این معماری، مقررات منتخب به عنوان زمینه در اختیار مدل زبانی قرار می‌گیرند و مدل موظف است پاسخ خود را بر اساس اطلاعات بازیابی شده تولید کند. همچنین برای کاهش احتمال تولید اطلاعات نادرست، در طراحی سامانه تأکید شده است که مدل در صورت ناکافی بودن منابع موجود، از ارائه اطلاعات خارج از زمینه خودداری کند.

در مجموع، پیاده‌سازی انجام شده نشان می‌دهد که ترکیب ساختار دهی داده‌های حقوقی، روش‌های مختلف بازیابی اطلاعات، بازرتبه بندی و مدل‌های زبانی بزرگ می‌تواند چارچوب مناسبی برای توسعه سامانه‌های پاسخگویی حقوقی فارسی فراهم کند. سامانه ایجاد شده علاوه بر فراهم کردن امکان پاسخ‌گویی مبتنی بر منابع حقوقی، زیرساخت لازم برای توسعه قابلیت‌های پیشرفته‌تر (مانند GraphRag) در حوزه تحلیل قوانین و اسناد حقوقی را نیز فراهم می‌کند.

۵-۲. محدودیت‌ها

در پیاده‌سازی سامانه چالش‌ها و محدودیت‌های مختلفی پیش رو آمد. مهم‌ترین محدودیت، کمبود زمان برای ارائه پروژه‌ای قوی‌تر و انقلابی در این حوزه بود. من از لپ‌تاپی استفاده می‌کردم که کارت گرافیک آن GTX-1650 بود و تنها چهار گیگ حافظه VRAM برای کارهای embedding و بازرتبه بندی در اختیار داشتم. به خاطر همین حجم کم VRAM امکان اجرای همزمان بازرتبه بندی و مدل embedding فراهم نبود و مجبور به اجرای همه در CPU بودم.

محدودیت دیگری که در طراحی سامانه وجود داشت، وابستگی کیفیت پاسخ جستجو (معنایی و بازرتبه بندی) به مدلی بود که در حال استفاده است. مدل‌های استفاده شده از کیفیت بالایی برخوردار هستند، اما نمی‌توان با قطعیت در مورد صحیح بودن ۱۰۰ درصد مطالب ارائه شده نظر داد.

محدودیت دیگر، عدم امکان اجرای قسمتی بود که ارتباط بین مقررات را کشف کند. برای آزمایش این قسمت من از دو مدل محلی qwen-3.5 با پارامترهای 4b و 2b استفاده کردم که به ترتیب کار استخراج **reference** و **action** را انجام دهند. در سامانه حدود سیصد هزار مقرره وجود دارد و جستجوی جمله به جمله برای یافتن ارتباط (تاکید می کنم، جمله به جمله) نیاز به زمانی حدود ۲۰۰ ساعت داشت که مدل های بالا همچنان در بعضی از موارد نتیجه نادرست تحویل می دادند.

شاید یکی از مهم ترین محدودیت ها در این حوزه، نبود یک **dataset** و **benchmark** برای آزمایش بهتر سامانه است. درست است که در این سامانه از مدل های آماده استفاده شده است، ولی برای ساخت مدل های اختصاصی برای همین سامانه، این محدودیت می تواند این تصمیم را دچار مشکل جدی کند.

۵. منابع و مراجع

- Project Code Availabe at: <https://github.com/GreenGOC/persian-legal-ai>
- ..Chen, W., et al. (2024). Large Language Models in Law: A Survey. AI Open
- Bokaei, Z., Magdy, W., & Webber, B. (2026). Farsi Natural Language Processing: A Survey. Information Processing & Management
- Safavi, T., & Kouhbanani, A. (2018). Hazm: A Python Library for Persian Natural Language Processing
- Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 333–389
- Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, 758–759
- Sentence Transformers Documentation. Cross-Encoder and Reranking Documentation
- PartAI/Tooka-Sbert-V2-Large. Persian Embeddings. Hugging Face Model Repository. Available at: <https://huggingface.co/heydariAI/persian-embeddings>
- Aghadavoud Jolfaei, S. (2025). Reranker XLM-RoBERTa-Large for Persian Scientific Retrieval. Hugging Face Model Repository. Available at: <https://huggingface.co/safora/reranker-xlm-roberta-large>
- Cohere, “Rerank API Documentation,” Cohere Documentation. [Official documentation](#)
- Milvus Documentation. Vector Database Documentation
- Django Documentation. Django Web Framework Documentation
- PostgreSQL Documentation. PostgreSQL Documentation
- Available at: <https://nezamat.ir> نظامات. مجموعه تنقیحی قوانین و مقررات.
- سامانه ملی قوانین و مقررات جمهوری اسلامی ایران. پایگاه قوانین و مقررات کشور.
- روزنامه رسمی جمهوری اسلامی ایران. پایگاه رسمی انتشار قوانین و مقررات کشور..